Proyecto Final
Trabajo Práctico de Clasificación con Machine Learning
Comparación de Modelos ML contra la Solución Ganadora del Meli Challenge 2019

Maribel Fraire
Maestría en Data Science

8 de marzo de 2026

Resumen

Este trabajo presenta un análisis comparativo de diferentes modelos de Machine Learning aplicados al problema de clasificación propuesto por Mercado Libre en la competencia de Kaggle (Meli Challenge 2019). El objetivo principal es evaluar y comparar el rendimiento de diversos modelos contra la solución ganadora.

Índice

1. Introducción **4**
1.1. Contexto del Problema 4
1.2. Objetivo General 4
1.3. Objetivos Específicos 4

2. Marco Teórico **4**
2.1. Conceptos Básicos de Machine Learning para Clasificación 4
2.1.1. Aprendizaje Automático 4
2.1.2. Tarea de Clasificación 5
2.1.3. Medida de Rendimiento 5
2.1.4. Generalización 5
2.1.5. Descenso por Gradiente Estocástico y Función de Pérdida 7

2.2. Modelos de Machine Learning para Clasificación de Texto 9
2.3. Arquitecturas de Redes Neuronales 10
2.4. Embeddings y modelos tipo Transformer 10
2.4.1. Tokenización 10
2.4.2. Embedding lookup (búsqueda de embeddings) 11
2.4.3. Codificación posicional (positional encoding) 11
2.4.4. Mecanismo de autoatención (self-attention) 11
2.4.5. Agregación y pooling 11
2.4.6. Red neuronal feed-forward 12

3. Metodología **12**
3.1. Dataset 12
3.2. Preprocesamiento 12
3.3. Modelos Evaluados 12
3.4. OpenAI Embeddings (Modelo 3) 13
3.4.1. Metodología del embedding (text-embedding-3-small) 13
3.4.2. Descripción de la solución 13
3.4.3. Ensayo 1 (03_tp_final_embedding.ipynb) 13
3.4.4. Ensayo 2 (03_tp_final_embedding_2.ipynb) 14
3.4.5. Batch API 14
3.5. BETO Aplicado al Dataset Completo en Español 15
3.5.1. Configuración del Entrenamiento 15
3.5.2. Dependencia de Accelerate 15
3.5.3. Preparación del Dataset 16
3.5.4. Requisitos de Software 16
3.5.5. Validación de Extrapolabilidad de Resultados 16

4. Problemas Técnicos Encontrados **17**
4.1. Limitaciones de la Solución Original 17
4.2. Workarounds Implementados 18

5. Resultados **19**
5.1. Métricas de Evaluación 19
5.2. Análisis Comparativo 20

6. Discusión **20**

7. Conclusiones **20**

8. Trabajo Futuro **20**

9. Referencias **20**

A. Código y Repositorio **21**

1. Introducción

1.1. Contexto del Problema

El Meli Challenge 2019 fue una competencia de Kaggle organizada por Mercado Libre con el objetivo de clasificar productos en diferentes categorías basándose en sus títulos y descripciones. La solución ganadora que alcanzó un score de 0.91733 en el Public Leaderboard, ocupando el primer lugar.

1.2. Objetivo General

Evaluar el rendimiento de enfoques de clasificación de texto que incluye embeddings clásicos, aprendizaje profundo y modelos basados en transformers, en el marco del Meli Challenge 2019.

1.3. Objetivos Específicos

1. Analizar las caracteristicas y el alcance de los modelos de clasificación de texto utilizados en la práctica: embeddings clásicos, aprendizaje profundo y modelos basados en la arquitectura Transformer o LLM.

2. Implementar pipelines completos de clasificación de texto que incluyan tokenización, obtención de embeddings, clasificación con diferentes soluciones aplicadas en la práctica tales como embeddings OpenAI (`text-embedding-3-small`), modelos preentrenados BETO/BERT y GPT, embeddings FastText con red neuronal y solución ganadora con LSTM/GRU.

3. Comparar las métricas de rendimiento de los modelos implementados y de la solución ganadora, para identificar factores que explican sus diferencias en el problema de categorización del Meli Challenge 2019.

2. Marco Teórico

2.1. Conceptos Básicos de Machine Learning para Clasificación

Esta sección describe algunos conceptos básicos del Machine Learning que son utilizados en este trabajo.

2.1.1. Aprendizaje Automático

Siguiendo el marco conceptual establecido por Mitchell (1997), se define el Machine Learning como: “A computer program is said to learn from experience E with respect to

some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E"[1]. Esta definición establece tres componentes fundamentales: la experiencia (E), que corresponde a los datos de entrenamiento; la tarea (T), que define qué tipo de problema se busca resolver; y la medida de rendimiento (P), que cuantifica qué tan bien el programa realiza la tarea.

2.1.2. Tarea de Clasificación

La clasificación es un tipo de tarea de aprendizaje supervisado en la cual el objetivo es asignar una etiqueta o categoría a una instancia de entrada basándose en sus características. En este tipo de tarea, el algoritmo aprende a partir de ejemplos etiquetados (experiencia E) para construir un modelo que pueda predecir la categoría correcta para nuevas instancias no vistas durante el entrenamiento. En el contexto del Meli Challenge 2019, la tarea consiste en clasificar productos de e-commerce en categorías específicas basándose en sus títulos y descripciones, donde cada producto representa una instancia y cada categoría representa una clase posible.

2.1.3. Medida de Rendimiento

La medida de rendimiento permite cuantificar el éxito del algoritmo en la tarea de clasificación. En este trabajo se utiliza principalmente la *accuracy* (precisión o exactitud), definida como la proporción de predicciones correctas sobre el total de predicciones realizadas. Matemáticamente, la accuracy se calcula como:

$$\text{Accuracy} = \frac{\text{Número de predicciones correctas}}{\text{Número total de predicciones}} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

donde TP (True Positives) son los casos correctamente clasificados como positivos, TN (True Negatives) son los casos correctamente clasificados como negativos, FP (False Positives) son los casos incorrectamente clasificados como positivos, y FN (False Negatives) son los casos incorrectamente clasificados como negativos. La accuracy proporciona una medida general del rendimiento del modelo, aunque puede ser engañosa en problemas con clases desbalanceadas, donde algunas categorías tienen muchos más ejemplos que otras.

2.1.4. Generalización

El desafío principal de un algoritmo de Machine Learning es performar bien en datos nuevos, es decir, no vistos durante el entrenamiento. Esta cualidad es conocida como *generalización*.

[1] Mitchell, T. M. (1997). *Machine Learning*. McGraw Hill.

Los factores que determinan qué tan bien funcionará un algoritmo de Machine Learning pueden dividirse en dos categorías principales[2]: 1) hacer que el training error sea pequeño, y 2) hacer que la brecha entre training error y test error sea pequeña. Estos dos factores corresponden a los dos desafíos centrales en Machine Learning: *underfitting* y *overfitting*.

El *underfitting* ocurre cuando el modelo no es capaz de obtener un training error suficientemente bajo. Esto sucede cuando la capacidad del modelo es insuficiente para capturar la complejidad subyacente de los datos. En el contexto de clasificación de productos del Meli Challenge 2019, un modelo con underfitting podría no aprender los patrones necesarios para distinguir entre categorías similares, como por ejemplo, diferenciar entre "celulares" y "tablets".

El *overfitting* ocurre cuando la brecha entre training error y test error es demasiado grande, es decir, cuando el modelo tiene un training error muy bajo pero un test error significativamente mayor. Esto sucede cuando el modelo tiene demasiada capacidad y memoriza los detalles específicos del conjunto de entrenamiento en lugar de aprender patrones generalizables. En el problema de clasificación de categorías de productos, un modelo con overfitting podría memorizar títulos específicos del conjunto de entrenamiento y fallar al clasificar productos similares pero no idénticos en el conjunto de test.

La *capacidad* de un modelo se refiere a su habilidad para ajustarse a una amplia variedad de funciones. Un modelo con poca capacidad puede tener dificultades para ajustarse al conjunto de entrenamiento (underfitting), mientras que un modelo con demasiada capacidad puede memorizar el conjunto de entrenamiento sin generalizar bien (overfitting). La capacidad debe estar alineada con la complejidad del problema: para el Meli Challenge 2019, que involucra más de 1,500 categorías diferentes con patrones semánticos complejos en títulos y descripciones de productos, se requieren modelos con suficiente capacidad para capturar estas relaciones, pero sin excederla de manera que se produzca overfitting.

La relación entre la capacidad del modelo y los errores de entrenamiento y generalización se ilustra en la Figura 1, donde se observa cómo el error de entrenamiento disminuye con el aumento de la capacidad, mientras que el error de generalización alcanza un mínimo en un punto óptimo y luego aumenta debido al overfitting.

[2]Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

capacidad_error.png

Figura 1: Relación entre la capacidad del modelo y los errores de entrenamiento y generalización. La zona de underfitting se caracteriza por altos errores en ambos conjuntos, mientras que la zona de overfitting muestra un bajo error de entrenamiento pero un alto error de generalización, con una brecha creciente entre ambos.

2.1.5. Descenso por Gradiente Estocástico y Función de Pérdida

El descenso de gradiente (GD) es un algoritmo de optimización que minimiza iterativamente una función objetivo. En el contexto del Machine Learning, esta técnica se utiliza durante la fase de entrenamiento de un modelo supervisado para minimizar la función de pérdida o costo, que mide qué tan lejos están las predicciones del modelo de los valores reales de los datos[3]. Un ejemplo común de función de costo es el error cuadrático medio.

El algoritmo calcula el gradiente, o la pendiente, de la pérdida con respecto a los parámetros del modelo. Con este gradiente, da un paso en la dirección opuesta para reducir la pérdida. La tasa de aprendizaje α es el tamaño de los pasos y permanece fija para todos los parámetros del modelo. Este proceso se repite iterativamente hasta que el modelo logra una convergencia cercana a un mínimo. El mínimo global es el valor absoluto

[3] IBM. *Stochastic Gradient Descent*. https://www.ibm.com/mx-es/think/topics/stochastic-gradient-descent

más bajo de la función de pérdida en todo su dominio, lo que representa la mejor solución posible para el problema. Si la tasa de aprendizaje no es lo suficientemente pequeña, el algoritmo a menudo convergerá en un mínimo local en lugar del mínimo global. Una tasa de aprendizaje bien elegida es esencial para minimizar la función de pérdida y lograr la convergencia en un mínimo global.

El uso de *Stochastic Gradient Descent* (SGD) en redes neuronales se debe al alto costo computacional de ejecutar la backpropagation sobre el conjunto de entrenamiento completo[4]. SGD puede superar este costo y, aun así, conducir a una convergencia rápida. A diferencia del descenso de gradiente estándar, que actualiza los parámetros θ de la función objetivo $J(\theta)$ usando todo el conjunto de entrenamiento. SGD calcula el gradiente de los parámetros utilizando solo una muestra o unas pocas muestras de entrenamiento (minibatch). La actualización de parámetros en SGD está dada por:

$$\theta = \theta - \alpha \nabla_\theta J(\theta; x^{(i)}, y^{(i)}) \tag{2}$$

donde $(x^{(i)}, y^{(i)})$ es un par del conjunto de entrenamiento. Generalmente, cada actualización de parámetros en SGD se calcula con respecto a unas pocas muestras de entrenamiento o un minibatch, en lugar de una sola muestra. Esto reduce la varianza en la actualización de parámetros y puede conducir a una convergencia más estable.

El proceso de optimización mediante SGD se ilustra en la Figura 2, donde se observa cómo el algoritmo itera desde un punto inicial, ajustando los parámetros del modelo (pesos) para reducir progresivamente la pérdida hasta alcanzar un mínimo.

[4]Stanford University. *Optimization: Stochastic Gradient Descent*. UFLDL Tutorial. http://deeplearning.stanford.edu/tutorial/supervised/OptimizationStochasticGradientDescent/

sgd_optimizacion.png

Figura 2: Proceso de optimización mediante descenso por gradiente estocástico.

2.2. Modelos de Machine Learning para Clasificación de Texto

La clasificación de texto es una tarea fundamental en el procesamiento de lenguaje natural (NLP) que consiste en asignar etiquetas o categorías a documentos de texto. En el contexto del Meli Challenge 2019, el objetivo es clasificar productos de e-commerce en categorías específicas basándose en sus descripciones.

En la literatura es habitual organizar los enfoques de clasificación de texto en tres grandes grupos. Los *enfoques tradicionales* incluyen representaciones como TF-IDF o FastText con clasificadores clásicos (por ejemplo, regresión logística, SVM o árboles). El *aprendizaje profundo* abarca redes recurrentes (LSTM, GRU) y convolucionales aplicadas al texto. Los *transformers* corresponden a arquitecturas basadas en mecanismos de atención (BERT, BETO, GPT). Esta taxonomía es la que se adopta en el presente trabajo para comparar modelos.

2.3. Arquitecturas de Redes Neuronales

Las arquitecturas de redes neuronales profundas han demostrado ser efectivas para tareas de clasificación de texto. Entre las más utilizadas se encuentran:

- **LSTM (Long Short-Term Memory):** Redes neuronales recurrentes capaces de capturar dependencias de largo plazo en secuencias de texto
- **GRU (Gated Recurrent Unit):** Variante más eficiente computacionalmente que LSTM
- **Transformers:** Arquitecturas basadas en mecanismos de atención que han revolucionado el NLP moderno
- **Embeddings:** Representaciones vectoriales densas de palabras o tokens que capturan relaciones semánticas

2.4. Embeddings y modelos tipo Transformer

Los embeddings de texto utilizados en clasificación, búsqueda y bases vectoriales suelen producirse con *encoders* basados en la arquitectura Transformer. Esta subsección resume los conceptos y etapas típicas de ese pipeline, siguiendo la exposición de Gutsch[5]. Estos conceptos sustentan, en mayor o menor medida, modelos como BERT y los embeddings de APIs como la de OpenAI utilizados en la etapa de laboratorio (Modelo 3).

2.4.1. Tokenización

La tokenización es el proceso de dividir el texto en unidades más pequeñas o *tokens*. Los tokens pueden ser palabras, subpalabras o caracteres. En modelos clásicos como Word2Vec se usaba vocabulario a nivel de palabra, lo que limita el manejo de palabras fuera de vocabulario (OOV) y hace crecer el modelo con el tamaño del vocabulario.

Entre las técnicas de *subpalabras* utilizadas en este proyecto cabe destacar dos familias. Por un lado, **FastText** (Modelo 4, FastText) define las subpalabras como *n-gramas de caracteres* de cada palabra (p. ej. "celular" → "cel", "elu", "lul", "ula", "lar"). El vector de la palabra se obtiene como la suma de los vectores de sus n-gramas; así se pueden representar palabras OOV que comparten n-gramas con el corpus de entrenamiento. Por otro lado, los modelos tipo Transformer (BERT, APIs como OpenAI) usan *tokenización por subpalabras* con esquemas como Byte Pair Encoding (BPE) o WordPiece: las palabras frecuentes se mantienen enteras y las raras se descomponen en subunidades, con vocabularios del orden de 30k–50k tokens (p. ej. en BERT o GPT-2). En ambos casos

[5] Gutsch, D. (2023). *The Art of Embeddings: Transforming Text for Vector Databases (Part 2)*. Medium. https://medium.com/@david.gutsch0/the-art-of-embeddings-transforming-text-for-vector-databases-926738443e70

se controla el tamaño del vocabulario y se mejora el manejo de OOV. La elección del esquema de tokenización afecta el tamaño del modelo, el manejo de OOV y la calidad final de los embeddings.

2.4.2. Embedding lookup (búsqueda de embeddings)

Una vez tokenizado el texto, cada token se mapea a un vector denso inicial mediante una tabla de embeddings (embedding lookup). Estos vectores no son aleatorios: se aprenden en la fase de pre-entrenamiento del modelo y constituyen el punto de partida para que las capas posteriores capturen el significado de cada token en contexto.

2.4.3. Codificación posicional (positional encoding)

La arquitectura Transformer no tiene noción intrínseca del orden de los tokens. Para incorporar la posición en la secuencia, se suman *codificaciones posicionales* a los embeddings iniciales. Así el modelo puede distinguir, por ejemplo, "casa blanca" de "blanca casa". Esta etapa permite procesar la secuencia de forma no secuencial (a diferencia de RNN/LSTM), lo que facilita el paralelismo y el entrenamiento a escala.

2.4.4. Mecanismo de autoatención (self-attention)

El mecanismo de autoatención permite que cada token "consulte" al resto de los tokens de la secuencia y pondere su influencia según la relevancia. Así se obtienen representaciones que dependen del contexto completo (p. ej. la misma palabra tiene distinto vector según la oración). En la práctica, para cada posición se calcula una combinación ponderada de las representaciones de todas las posiciones; los pesos se obtienen a partir de productos escalares entre vectores de consulta y clave (attention scores), normalizados típicamente con softmax.

2.4.5. Agregación y pooling

Dentro de una capa Transformer, la salida del mecanismo de atención se agrega (p. ej. se suma con residual y se normaliza) y se obtiene un vector por token que ya incorpora contexto. Para tareas que requieren *un único vector por documento o por título* (como clasificación o búsqueda por similitud), se aplica una etapa adicional de *pooling* sobre los vectores de los tokens: por ejemplo, promediar los vectores de todos los tokens (mean pooling), o usar el vector asociado a un token especial como `[CLS]` en BERT. Ese vector único es el embedding que se indexa o se usa como entrada a un clasificador.

2.4.6. Red neuronal feed-forward

Tras las operaciones de atención (y opcionalmente más capas apiladas), los vectores por token se pasan por una red neuronal feed-forward (capas lineales con no linealidad), compartida para todas las posiciones. Esta red refina las representaciones y contribuye a la capacidad del modelo de capturar patrones más abstractos. La salida final de la última capa (antes o después del pooling, según el diseño) es el embedding que se utiliza para comparación o clasificación.

En resumen, el flujo típico es: *tokenización* → *embedding lookup* → + *codificación posicional* → (repetido) *autoatención* + *agregación/residual* + *feed-forward* → *pooling* (si se requiere un vector por texto). Los modelos de embedding como `text-embedding-3-small` de OpenAI no divulgan su arquitectura interna, pero se apoyan en principios análogos para producir vectores densos a partir de texto.

3. Metodología

3.1. Dataset

El dataset utilizado corresponde al proporcionado por Mercado Libre para la competencia Meli Challenge 2019. Este contiene información de productos de e-commerce con sus respectivas categorías como variable objetivo.

3.2. Preprocesamiento

El preprocesamiento de texto es crucial para el rendimiento de los modelos. La solución ganadora implementó técnicas específicas de normalización, incluyendo la normalización de valores numéricos a representaciones consistentes sin eliminarlos completamente, lo cual demostró ser una estrategia exitosa.

3.3. Modelos Evaluados

Se evaluaron los siguientes modelos como alternativas y comparación con la solución ganadora:

- **Modelo 1:** Basado en embeddings de OpenAI GPT (notebook: 01_tp_final_openai_gpt_oss.ipynb)
- **Modelo 2:** Basado en BETO (BERT pre-entrenado para español, notebook: 02_tp_final_beto.ipynb 02_tp_final_beto_all.ipynb)
- **Modelo 3:** Basado en embeddings de OpenAI (text-embedding-3-small). Dos ensayos: 03_tp_final_embedding.ipynb (Ensayo 1) y 03_tp_final_embedding_2.ipynb (Ensayo 2, Batch API).

- **Modelo 4:** Basado en FastText con redes neuronales (notebook: 04_tp_final_ft_nn.ipynb)
- **Solución Original:** Modelo ganador del 2019 (Eduardo García, GitHub: eduagarcia/meli-challenge-2019)

3.4. OpenAI Embeddings (Modelo 3)

Se utilizó el modelo `text-embedding-3-small` de OpenAI para el problema de clasificación de categorías de productos. Esta sección describe la metodología del embedding, la solución implementada, los dos ensayos realizados y el uso de la Batch API.

3.4.1. Metodología del embedding (text-embedding-3-small)

Según la documentación de OpenAI[6], los embeddings son representaciones numéricas de texto que permiten medir la afinidad semántica entre dos fragmentos. El modelo `text-embedding-3-small` es una versión mejorada y más eficiente del modelo anterior `ada`; está pensado para búsqueda, clustering, recomendaciones, detección de anomalías y tareas de clasificación.

La API de embeddings recibe como entrada el texto (en este caso, el título del producto) y devuelve un único vector denso de dimensión fija (1536 para `text-embedding-3-small`). Ese vector resume el contenido semántico del texto en una representación de tamaño fijo apta para entrenar clasificadores estándar. Se utiliza la API para obtener el vector de embedding para cada ejemplo mediante llamadas síncronas. La documentación pública no detalla el pipeline interno (p. ej. tokenización, representación por subpalabras o agregación en un único vector).

3.4.2. Descripción de la solución

La solución consiste en: (1) obtener un vector de embedding por título mediante la API de OpenAI; (2) usar esos vectores como features para entrenar clasificadores supervisados. Se probaron Random Forest y K-Nearest Neighbors (KNN), con variantes de KNN: sin escalado, con StandardScaler y con reducción de dimensionalidad (PCA a 50 componentes). La evaluación se realiza sobre un conjunto de test estratificado; las métricas reportadas son accuracy (y, donde aplica, classification report).

3.4.3. Ensayo 1 (03_tp_final_embedding.ipynb)

- **Muestra:** 1.000 registros tomados al azar del dataset; filtro de al menos 3 ejemplos por categoría, resultando en 247 filas útiles.
- **API:** Llamadas síncronas a la API de embeddings (no Batch).

[6] https://developers.openai.com/api/docs/models/text-embedding-3-small

- **Train/valid:** 70 % entrenamiento, 30 % validación (172 y 75 muestras respectivamente).
- **Mejor resultado:** KNN con escalado (StandardScaler), accuracy 0,53 en validación.

3.4.4. Ensayo 2 (03_tp_final_embedding_2.ipynb)

- **Muestra:** 10.000 registros con muestreo estratificado por categoría; filtro técnico de al menos 20 ejemplos por categoría (alineado con 02_tp_final_beto_all) y luego filtro de al menos 2 ejemplos por clase para permitir split estratificado (70 % train / 30 % test).
- **API:** Batch API de OpenAI: mismo modelo y misma lógica de embedding, con procesamiento asíncrono y descuento del 50 % en coste[7].
- **Train/test:** 70 % entrenamiento, 30 % test (~6.880 y ~2.949 muestras).
- **Mejor resultado:** KNN sin escalado, accuracy 0,48 en test; Random Forest 0,19; KNN con PCA 0,38.

La accuracy no mejoró en el Ensayo 2 a pesar de usar una muestra mayor (10k frente a ~247) porque el problema es más difícil: en el Ensayo 2 se conservan muchas más categorías (con mínimo 20 ejemplos en el dataset completo y luego 2 en la muestra), de modo que el número de clases es mucho mayor. A mayor número de clases, la tarea de clasificación es más difícil y la accuracy típicamente baja para el mismo modelo y la misma métrica; además, el Ensayo 1 evaluaba en un conjunto de validación pequeño y con pocas categorías, por lo que la comparación directa no es equitativa. La metodología del Ensayo 2 (Batch API, split train/test, filtros por categoría) es más rigurosa para reportar resultados.

3.4.5. Batch API

La Batch API de OpenAI[8] permite enviar conjuntos de peticiones de forma asíncrona. Para el endpoint `/v1/embeddings`, los parámetros de cada petición (modelo, texto de entrada) son los mismos que en la API síncrona; la diferencia está en el modo de envío y el coste. Ventajas relevantes para este proyecto: (1) descuento del 50 % respecto a la API estándar; (2) límites de tasa separados y más altos; (3) tiempo de completitud dentro de 24 horas (suele ser menor). Cada lote se prepara en un archivo JSONL (una línea por petición, con `custom_id`, `method`, `url` y `body`); el archivo se sube con `purpose="batch"` y

[7] https://developers.openai.com/api/docs/guides/batch
[8] https://developers.openai.com/api/docs/guides/batch

se crea el batch indicando `endpoint=/v1/embeddings"`. Los resultados se descargan por `output_file_id`; el orden de las líneas puede no coincidir con el de entrada, por lo que se usa `custom_id` para asociar cada respuesta al título correspondiente. Para embeddings, el límite es de 50.000 entradas por lote; en el Ensayo 2 se usaron 10.000 títulos en un solo lote.

3.5. BETO Aplicado al Dataset Completo en Español

Para evaluar el rendimiento del modelo BETO en condiciones más representativas, se implementó una versión que utiliza el dataset completo en español (~10 millones de registros) en lugar de las muestras reducidas utilizadas en experimentos preliminares.

3.5.1. Configuración del Entrenamiento

El entrenamiento se realizó en un equipo con GPU NVIDIA RTX 3060 (12GB VRAM), lo cual requirió configuraciones específicas para optimizar el uso de memoria y acelerar el proceso:

- **Mixed Precision (FP16):** Reducción del uso de VRAM en aproximadamente 40 % y aceleración del entrenamiento en ~20 %
- **Batch size:** 16 por dispositivo con gradient accumulation de 2 pasos (batch efectivo de 32)
- **Warmup ratio:** 10 % de los steps totales para estabilizar el entrenamiento inicial
- **Early stopping:** Paciencia de 2 epochs para evitar overfitting

3.5.2. Dependencia de Accelerate

Un aspecto técnico relevante es la dependencia de la librería `accelerate` de Hugging Face. El `Trainer` de Hugging Face Transformers delega todas las operaciones de bajo nivel a esta librería, incluyendo:

- Movimiento de tensores entre CPU y GPU
- Implementación de mixed precision (FP16/BF16)
- Manejo de gradient accumulation
- Soporte para entrenamiento distribuido multi-GPU

Esta separación arquitectónica se realizó para mantener la librería `transformers` más liviana, permitir configuraciones avanzadas sin modificar el código base, y reutilizar la lógica de aceleración en otras librerías del ecosistema Hugging Face (como `diffusers`).

3.5.3. Preparación del Dataset

Para el split estratificado del dataset se requirió un filtro mínimo de 20 ejemplos por categoría. Esto se debe a que el proceso de división (90 % train, 5 % validación, 5 % test) con estratificación necesita suficientes ejemplos en cada categoría para garantizar representación en todos los conjuntos. Categorías con menos de 20 ejemplos pueden resultar en splits donde alguna categoría quede con solo 1 ejemplo, causando errores en la estratificación.

3.5.4. Requisitos de Software

La ejecución en GPU moderna (RTX 30xx/40xx) con CUDA 12.x requiere:

- PyTorch $\geq$ 2.6 (debido a vulnerabilidad de seguridad CVE-2025-32434 en versiones anteriores)
- `accelerate` $\geq$ 0.26.0
- Drivers NVIDIA actualizados compatibles con CUDA 12.x

3.5.5. Validación de Extrapolabilidad de Resultados

Con el propósito de validar si los resultados obtenidos con una muestra reducida de 70,000 registros (notebook `02_tp_final_beto.ipynb`) eran extrapolables al dataset completo en español, se entrenó el mismo modelo BETO con el dataset completo de aproximadamente 10 millones de registros (notebook `02_tp_final_beto_all.ipynb`) utilizando la misma arquitectura y configuración de hiperparámetros.

Los resultados obtenidos muestran una diferencia en el rendimiento entre ambas configuraciones:

- **Muestra de 70,000 registros:** Accuracy de 0.46 en el conjunto de test
- **Dataset completo (~10M registros):** Accuracy de 0.88 en el conjunto de test

La diferencia observada de 42 puntos porcentuales indica que los resultados no son extrapolables de la muestra pequeña al dataset completo. Las posibles razones de esta discrepancia incluyen:

1. **Representatividad de la muestra:** Una muestra de 70,000 registros puede no capturar adecuadamente la distribución completa de categorías y patrones presentes en el dataset completo, considerando que el dataset original contiene más de 1,500 categorías diferentes.

2. **Balance de clases:** Con una muestra pequeña, algunas categorías pueden estar subrepresentadas o ausentes, afectando la capacidad del modelo para aprender patrones generalizables.

3. **Overfitting en muestra pequeña:** Con menos datos, el modelo puede memorizar patrones específicos de la muestra en lugar de aprender representaciones generalizables, resultando en un rendimiento reducido cuando se evalúa en el conjunto completo.

4. **Capacidad del modelo:** BETO es un modelo de 110 millones de parámetros que requiere grandes cantidades de datos para alcanzar su potencial. Con 70,000 ejemplos, el modelo no tiene suficientes datos para aprender efectivamente las relaciones complejas entre texto y categorías.

Esta validación demuestra la importancia de evaluar modelos en datasets de tamaño similar al de producción, especialmente cuando se trabaja con modelos transformer de gran capacidad como BETO. Los resultados preliminares con muestras pequeñas pueden no reflejar el rendimiento del modelo en condiciones de producción.

4. Problemas Técnicos Encontrados

4.1. Limitaciones de la Solución Original

La solución ganadora del 2019 presenta problemas de reproducibilidad debido a:

1. **Dependencias obsoletas:**
   - Python 3.6 / 3.7 (EOL - End of Life)
   - TensorFlow 1.14
   - CUDA 10.0 + cuDNN 7.6

2. **Incompatibilidades técnicas:**
   - TensorFlow 1.14 no es compatible con GPUs modernas (RTX 30xx)
   - CuDNNLSTM y CuDNNGRU requieren versiones específicas de CUDA/cuDNN que ya no están disponibles para drivers modernos
   - Error: "No OpKernel was registered to support Op 'CudnnRNN"
   - Estas capas no tienen fallback a CPU, por lo que el entrenamiento falla inmediatamente sin GPU funcional

3. **Problemas con bibliotecas:**

- Incompatibilidad gensim + smart_open
- SyntaxError con Python 3.6 por uso de `from __future__ import annotations`
- El paquete smart_open utiliza anotaciones de tipo de Python 3.10+, no soportadas en Python 3.6

4. **Recursos computacionales:**
   - Embeddings de gran tamaño (~6 GB)
   - FastText requiere mucha RAM durante el entrenamiento
   - Entrenamiento extremadamente lento en CPU
   - Costo computacional elevado incluso con las tecnologías modernas

4.2. Workarounds Implementados

Para poder evaluar los modelos, se implementaron las siguientes soluciones alternativas:

- Reemplazo de CuDNNLSTM/CuDNNGRU por LSTM/GRU estándar (permitiendo ejecución en CPU pero perdiendo la aceleración GPU específica de la solución original)
- Migración a TensorFlow 2.x o alternativas modernas
- Adaptación del código para ejecución en CPU
- Uso de versiones modernas de Python (3.8+) y bibliotecas compatibles

Nota importante: Estos workarounds permiten la ejecución funcional pero alteran la arquitectura original, haciendo imposible una replicación exacta de la solución ganadora del 2019 en entornos modernos.

5. Resultados

5.1. Métricas de Evaluación

Cuadro 1: Comparación de Métricas por Modelo

Modelo	Descripción	Accuracy	Precision	Recall	F1-Score
Solución Original (2019)	Dataset completo, modelo ganador del Meli Challenge 2019	0.917	-	-	-
Modelo 1 (GPT OSS)	Muestra de 1,000 registros, embeddings de OpenAI GPT	0.91	-	-	0.89
Modelo 2 (BETO)	Muestra de 70,000 registros en español, BETO (BERT preentrenado)	0.47	-	-	-
Modelo 2 (BETO completo)	Dataset completo en español (˜10M registros), BETO	0.88	-	-	0.84
Modelo 3 (Embeddings v1)	Muestra 1k → 247 filas, text-embedding-3-small, API síncrona, KNN escalado	0.53	-	-	-
Modelo 3 (Embeddings v2)	Muestra 10k estratificada, text-embedding-3-small, Batch API, KNN (mejor)	0.48	-	-	-
Modelo 4 (FastText NN)	Dataset completo en español (˜10M registros), FastText con redes neuronales	0.81	-	-	-

Nota: Los valores "-" indican métricas no reportadas en el notebook correspondiente (precision/recall no calculados en todos los experimentos).

5.2. Análisis Comparativo

6. Discusión

7. Conclusiones

8. Trabajo Futuro

9. Referencias

Referencias

[1] Mitchell, T. M. (1997). *Machine Learning*. McGraw Hill.

[2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[3] IBM. *Stochastic Gradient Descent*. IBM Think Topics. https://www.ibm.com/mx-es/think/topics/stochastic-gradient-descent

[4] Stanford University. *Optimization: Stochastic Gradient Descent*. UFLDL Tutorial. http://deeplearning.stanford.edu/tutorial/supervised/OptimizationStochasticGradientDescent/

[5] Mercado Libre. (2019). *Meli Challenge 2019*. Kaggle Competition. https://www.kaggle.com/c/mercadolibre-data-challenge-2019

[6] García, E. (2019). *MeLi Data Challenge 2019 - Winner Solution*. GitHub Repository. https://github.com/eduagarcia/meli-challenge-2019

[7] TensorFlow Team. *TensorFlow Documentation*. https://www.tensorflow.org/

[8] OpenAI. *Batch API*. API Documentation. https://developers.openai.com/api/docs/guides/batch

[9] OpenAI. *text-embedding-3-small*. API Documentation, Models. https://developers.openai.com/api/docs/models/text-embedding-3-small

[10] Mike H. (s.f.). *Tutorial: Batch embedding with OpenAI API*. Medium. https://medium.com/@mikehpg/tutorial-batch-embedding-with-openai-api-95da95c9778a

[11] Gutsch, D. (2023). *The Art of Embeddings: Transforming Text for Vector Databases (Part 2)*. Medium. https://medium.com/@david.gutsch0/the-art-of-embeddings-transforming-text-for-vector-databases-926738443e70

[12] Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*, 5, 135–146. https://aclanthology.org/Q17-1010/

A. Código y Repositorio

El código completo, notebooks y scripts están disponibles en:

Nota: Actualiza esta URL con el enlace a tu repositorio si lo tienes disponible.